

ОТЧЁТ

по лабораторной работе № 2

**КОГДА ИНДЕКСОВ НЕДОСТАТОЧНО: ПРОИЗВОДИТЕЛЬНОСТЬ
POSTGRESQL ПРИ РОСТЕ ДАННЫХ**

по модулю «Масштабирование баз данных»

Исполнитель:

обучающийся группы K0709-23/3

подпись, дата

У. И. Усачева

Руководитель:

преподаватель

подпись, дата

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Исследование роста данных.....	7
1.1 Задания 1–2. Таблица events и генерация данных	7
1.2 Задание 3. Размер таблицы.....	7
1.3 Задание 4. SELECT без дополнительного индекса.....	8
1.4 Задание 5. Добавление индекса	8
1.5 Задание 6. Поиск по диапазону дат	10
1.6 Задание 7. Фильтрация и сортировка.....	10
1.7 Задание 8. Агрегация	12
1.8 Задание 9. Стоимость индексов.....	12
1.9 Задание 10. Размер индексов	14
1.10 Задание 11. Когда индекс уже не спасает.....	14
2 Исследование собственного сервиса TaskManager.....	17
2.1 Задание 12. Выбор сущности.....	17
2.2 Задание 13. Выбор запросов	17
2.3 Задание 14. Baseline	18
2.4 Задание 15. Увеличение объёма данных	19
2.5 Задание 16. Сравнение производительности.....	19
2.6 Задание 17. Узкое место	20
2.7 Задание 18. Попытка улучшить запрос.....	21
2.8 Задание 19. Граница применимости индексов.....	24
3 Ответы на контрольные вопросы	25
ЗАКЛЮЧЕНИЕ	28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30
ПРИЛОЖЕНИЕ А Скрипт измерений одного объёма данных	31
ПРИЛОЖЕНИЕ Б Миграция с составным индексом для фильтра по сроку..	33

ВВЕДЕНИЕ

Работа продолжает лабораторную работу № 1, в которой изучались индексы и `EXPLAIN ANALYZE`. Здесь исследуется следующий вопрос: что происходит с базой данных, когда объём данных увеличивается в десятки и сотни раз.

Цель работы — на измерениях определить, как рост объёма данных влияет на время выполнения запросов, планы выполнения, размер таблиц и индексов, операции записи, и найти момент, когда индексов становится недостаточно.

Для достижения цели решались следующие задачи:

- сгенерировать экспериментальную таблицу событий объёмом от 10 тысяч до 10 миллионов строк;
- измерить размер таблицы и индексов, время поиска, диапазонных, сортирующих и агрегирующих запросов на каждом объёме без индексов и с индексами;
- оценить стоимость индексов при вставке, изменении и удалении данных;
- провести аналогичное исследование реальных запросов сервиса TaskManager на 100 тысячах, 1 миллионе и 5 миллионах задач, найти узкое место и оптимизировать его;
- на основе измерений определить границу применимости индексов.

Работа состоит из двух частей: в части А эксперименты выполняются над таблицей `events` в отдельной схеме базы данных, в части В — над таблицей `tasks` сервиса. При выполнении использовались документация PostgreSQL [1–5], методические указания [6] и инструмент записи терминала VHS [7].

Таблица 1 – Среда выполнения экспериментов

Параметр	Значение
СУБД	PostgreSQL 16.14 (postgres:16-alpine) в Docker Desktop, Windows 11

Параметр	Значение
Ресурсы Docker	20 CPU, 10,7 ГБ памяти
shared buffers / work mem	128 МБ / 4 МБ
max_parallel_workers_per_gather	2
shm_size контейнера	512 МБ (см. задание 11)

Все эксперименты автоматизированы:

Таблица 2 – Скрипты и результаты

Часть	Запуск	Результаты
А — таблица events в схеме lab02	bash scripts/lab-02/run-part-a.sh (скрипты a0–a3)	docs/lab-02/results/a*.txt (планы), a-*.csv (сводные таблицы)
В — таблица tasks сервиса	RESET=1 bash scripts/lab-02/run-part-b.sh (b1-measure.sql), затем b2-fixes.sql	docs/lab-02/results/b*.txt, b-*.csv
Запись терминала	vhs scripts/lab-02/demo.tape	docs/lab-02/demo.gif

Как измерялось. Каждый замер делает функция `lab02.measure (scripts/lab-02/a0-setup.sql)`:

- 1) выполняет запрос вхолостую, чтобы данные попали в кэш;
- 2) получает план в формате JSON и извлекает из него тип сканирования, использованные индексы, actual rows, Rows Removed by Filter и наличие Sort;
- 3) выполняет EXPLAIN (ANALYZE, BUFFERS) ещё раз, печатает план в лог и сохраняет его Execution Time.

Все замеры лежат в таблице `lab02.measurements`, из неё выгружаются CSV. Время — один прогон на «тёплом» кэше: абсолютные значения зависят от машины, сравнивать нужно порядок величин и характер роста.

Запись терминала. GIF снят инструментом VHS: живой `psql` на таблице из 10 млн строк. Поиск без индекса, построение индекса, тот же поиск с индексом и агрегация за год, которую индекс не ускоряет.

```

taskmanager_db=# \timing on
Timing is on.
taskmanager_db=# SELECT count(*) FROM lab02.events;
 count
-----
10000000
(1 row)

Time: 194.211 ms
taskmanager_db=# EXPLAIN ANALYZE SELECT * FROM lab02.events WHERE user_id = 123;
                                QUERY PLAN
-----
 Gather  (cost=1000.00..136427.33 rows=100 width=36) (actual time=11.753..144.425 rows=103 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    -> Parallel Seq Scan on events  (cost=0.00..135417.33 rows=42 width=36) (actual time=8.860..119.734 rows=34 loops=3)
         Filter: (user_id = 123)
         Rows Removed by Filter: 3333299
 Planning Time: 0.175 ms
 JIT:
   Functions: 6
   Options: Inlining false, Optimization false, Expressions true, Deforming true
   Timing: Generation 0.721 ms, Inlining 0.000 ms, Optimization 0.758 ms, Emission 9.301 ms, Total 10.781 ms
 Execution Time: 144.636 ms
(12 rows)

Time: 145.795 ms
taskmanager_db=#

```

Рисунок 1 – Поиск по user_id на 10 млн строк без индекса: Parallel Seq Scan,
145 мс

```

taskmanager_db=# CREATE INDEX idx_events_user_id ON lab02.events (user_id);
CREATE INDEX
Time: 3995.108 ms (00:03.995)
taskmanager_db=# EXPLAIN ANALYZE SELECT * FROM lab02.events WHERE user_id = 123;
                                QUERY PLAN
-----
 Bitmap Heap Scan on events  (cost=5.21..396.07 rows=100 width=36) (actual time=0.134..1.092 rows=103 loops=1)
    Recheck Cond: (user_id = 123)
    Heap Blocks: exact=102
    -> Bitmap Index Scan on idx_events_user_id  (cost=0.00..5.18 rows=100 width=0) (actual time=0.106..0.107 rows=103 loops=1)
         Index Cond: (user_id = 123)
 Planning Time: 0.290 ms
 Execution Time: 1.160 ms
(7 rows)

Time: 2.121 ms
taskmanager_db=#

```

Рисунок 2 – Построение индекса idx_events_user_id (4 с) и тот же поиск:
Bitmap Index Scan, 1,2 мс

```

taskmanager_db=# EXPLAIN ANALYZE SELECT DATE(created_at), COUNT(*) FROM lab02.events WHERE created_at >= NOW() - INTERVAL '365 days' GROUP BY 1;

               QUERY PLAN
-----
HashAggregate  (cost=845828.24..1048951.25 rows=9999902 width=12) (actual time=3041.400..3041.801 rows=366 loops=1)
  Group Key: date(created_at)
  Planned Partitions: 128   Batches: 1   Memory Usage: 3121kB
  -> Seq Scan on events  (cost=0.00..283333.76 rows=9999902 width=4) (actual time=171.583..2168.434 rows=9999893 loops=1)
    Filter: (created_at >= (now() - '365 days'::interval))
    Rows Removed by Filter: 107
  Planning Time: 0.508 ms
  JIT:
    Functions: 8
    Options: Inlining true, Optimization true, Expressions true, Deforming true
    Timing: Generation 2.110 ms, Inlining 88.985 ms, Optimization 45.923 ms, Emission 36.614 ms, Total 173.632 ms
  Execution Time: 3046.542 ms
(12 rows)

Time: 3048.690 ms (00:03.049)
taskmanager_db=#

```

Рисунок 3 – Агрегация за год на 10 млн строк: Seq Scan и HashAggregate, 3 с
— индекс не помогает

1 Исследование роста данных

1.1 Задания 1–2. Таблица events и генерация данных

Таблица `events` создана по методичке в отдельной схеме `lab02` базы сервиса. Данные генерируются запросом из методички (функция `lab02.generate_events`). Таблица растёт до всех контрольных объёмов, включая необязательные 10 млн строк:

Таблица 3 – Время вставки при росте таблицы (индексов, кроме РК, нет)

Шаг роста	Вставлено строк	Время	На одну строку
0 → 10 000	10 000	0,035 с	3,5 мкс
10 000 → 100 000	90 000	0,23 с	2,6 мкс
100 000 → 1 000 000	900 000	2,37 с	2,6 мкс
1 000 000 → 5 000 000	4 000 000	10,28 с	2,6 мкс
5 000 000 → 10 000 000	5 000 000	12,29 с	2,5 мкс

Без дополнительных индексов вставка масштабируется линейно: ~2,5 мкс на строку при любом размере таблицы.

Особенность генератора: каждая ветка `CASE WHEN random() < ...` вызывает `random()` заново. Поэтому распределение типов не 40/30/20/10 %, а примерно MESSAGE 40 %, LOGIN 42 % ($0,6 \times 0,7$), PURCHASE 16 % ($0,6 \times 0,3 \times 0,9$), OTHER 2 %. На выводы работы это не влияет.

1.2 Задание 3. Размер таблицы

Таблица 4 – Размер таблицы events на разных объёмах

Количество строк	Размер таблицы	Общий размер (таблица + РК)	Общий размер с тремя индексами
10 000	672 кБ	952 кБ	1,7 МБ
100 000	6,5 МБ	8,7 МБ	15,8 МБ
1 000 000	65,1 МБ	86,6 МБ	147,0 МБ
5 000 000	325,5 МБ	432,8 МБ	724,3 МБ
10 000 000	651,0 МБ	865,5 МБ	1,41 ГБ

Размер растёт строго линейно: ~68 байт на строку в таблице плюс ~22 байта на строку в первичном ключе. Три дополнительных индекса занимают почти столько же, сколько сама таблица: 646 МБ против 651 МБ на 10 млн строк. Все объёмы, включая 10 млн, созданы успешно.

1.3 Задание 4. SELECT без дополнительного индекса

Листинг 1 – SQL-запрос (задание 4)

```
SELECT * FROM events WHERE user_id = 123;
```

Таблица 5 – Поиск по user_id без индекса

Количество строк	Scan	Actual Rows	Rows Removed	Execution Time
10 000	Seq Scan	0	10 000	0,32 мс
100 000	Seq Scan	0	100 000	3,58 мс
1 000 000	Parallel Seq Scan (2 workers)	15	999 987	19,7 мс
5 000 000	Parallel Seq Scan	57	4 999 944	93,7 мс
10 000 000	Parallel Seq Scan	102	9 999 897	161,6 мс

Листинг 2 – План на 10 млн строк (первый кадр GIF)

```
Gather (cost=1000.00..136427.33 rows=100 width=36) (actual time=11.753..144.425 rows=103 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  -> Parallel Seq Scan on events (cost=0.00..135417.33 rows=42 width=36) (actual time=8.860..119.734 rows=34 loops=3)
    Filter: (user_id = 123)
    Rows Removed by Filter: 3333299
Execution Time: 144.636 ms
```

Как изменилось время? Росло пропорционально объёму: на каждые 1 000 строк нужных — 1 000 000 проверенных, и все они отбрасываются фильтром. От 10 тыс. до 10 млн строк (в 1 000 раз) время выросло в 511 раз. Рост немного медленнее линейного только потому, что с 1 млн строк PostgreSQL подключает два параллельных процесса.

Вывод. Без индекса стоимость поиска определяется размером таблицы, а не размером результата. Строк в ответе становится больше (0 → 102), но отбрасывается в 100 000 раз больше. Такой запрос деградирует вместе с ростом данных.

1.4 Задание 5. Добавление индекса

Листинг 3 – SQL-запрос (задание 5)

```
CREATE INDEX idx_events_user_id ON events(user_id);
```

Листинг 4 – План выполнения (задание 5)

```
Bitmap Heap Scan on events (cost=5.21..396.07 rows=100 width=36) (actual time=0.134..1.092 rows=103 loops=1)
  Recheck Cond: (user_id = 123)
```

```

Heap Blocks: exact=102
-> Bitmap Index Scan on idx_events_user_id (cost=0.00..5.18 rows=100 width=0) (actual
time=0.106..0.107 rows=103 loops=1)
    Index Cond: (user_id = 123)
Execution Time: 1.160 ms

```

Таблица 6 – Поиск по user_id без индекса и с индексом

Объём	Без индекса	С индексом	Разница	Строк обработано с индексом
10 000	0,32 мс (Seq Scan)	0,019 мс (Index Scan)	в 17 раз	0
100 000	3,58 мс (Seq Scan)	0,035 мс (Bitmap)	в 102 раза	0
1 000 000	19,7 мс (Parallel Seq Scan)	0,050 мс (Bitmap)	в 393 раза	14
5 000 000	93,7 мс (Parallel Seq Scan)	0,097 мс (Bitmap)	в 966 раз	57
10 000 000	161,6 мс (Parallel Seq Scan)	0,159 мс (Bitmap)	в 1 016 раз	103

1) **Как изменился план?** Вместо полного прохода по таблице — Bitmap Index Scan по индексу с условием Index Cond: (user_id = 123) и чтение только тех страниц, где лежат найденные строки (Heap Blocks: exact=102). Rows Removed by Filter исчез: лишние строки не читаются вообще.

2) **Как изменилось время?** С сотен миллисекунд до десятых долей. Время с индексом растёт только вместе с числом найденных строк (0 → 103), а не с размером таблицы.

3) **На каком объёме разница стала заметной?** Относительная разница — уже на 100 тыс. строк (в 100 раз). Абсолютная, ощутимая для API, — с 1 млн: 20 мс на запрос без индекса при 1 000 запросов в секунду — это 20 секунд процессорного времени каждую секунду.

4) **Всегда ли индекс оптимален?** Нет:

- на 10 тыс. строк обе версии укладываются в доли миллисекунды;
- когда запрос возвращает большую часть таблицы, индекс не используется (задание 6);
- индекс занимает место и замедляет запись в 2,6–4 раза (задание 9).

1.5 Задание 6. Поиск по диапазону дат

Таблица 7 – Диапазон created_at за последние сутки и за полгода

Объём	Сутки без индекса	Сутки с индексом	Строк за сутки	Полгода без индекса	Полгода с индексом	Строк за полгода
10 000	1,23 мс	0,046 мс	18	1,67 мс	1,83 мс (Seq Scan)	4 869
100 000	15,4 мс	0,205 мс	266	17,1 мс	17,5 мс (Seq Scan)	49 674
1 000 000	52,3 мс	2,20 мс	2 720	165,7 мс	175,7 мс (Seq Scan)	493 321
5 000 000	266,3 мс	13,3 мс	13 697	873,5 мс	884,3 мс (Seq Scan)	2 465 383
10 000 000	479,4 мс	75,3 мс	27 229	1 824,0 мс	1 798,8 мс (Seq Scan)	4 931 646

Для суток индекс используется всегда (Bitmap Heap Scan + Bitmap Index Scan on idx_events_created_at) и ускоряет запрос в 6–27 раз. Для полугода (49 % таблицы) планировщик индекс не выбирает: Seq Scan с индексом и без него.

Почему индекс хорош для небольшого диапазона и бесполезен для большей части таблицы?

– Индекс сокращает работу пропорционально доле найденных строк: чтобы найти 0,27 % таблицы, достаточно прочитать 0,27 % индекса и нужные страницы таблицы.

– Каждая найденная строка — это переход в таблицу, часто на другую страницу. При 27 тыс. строк за сутки на 10 млн bitmap читает 23 тыс. страниц. Поэтому даже индексный запрос стал в 34 раза дороже, чем на 1 млн: данных за сутки стало больше.

– Когда нужна половина таблицы, почти каждая страница всё равно будет прочитана. Последовательное чтение всех страниц дешевле, чем тот же объём вразнобой плюс чтение индекса.

1.6 Задание 7. Фильтрация и сортировка

Листинг 5 – SQL-запрос (задание 7)

```
SELECT * FROM events WHERE user_id = 123 ORDER BY created_at DESC LIMIT 100;
```

Таблица 8 – Фильтрация по user_id с сортировкой по created_at и LIMIT 100

Объём	Без индекса	Индекс (user_id)	Индексы (user_id) + (user_id, created_at DESC)	Только (user_id, created_at DESC)
10 000	Seq Scan → Sort , 0,38 мс	Index Scan → Sort , 0,083 мс	Index Scan по составному, без Sort , 0,019 мс	—
100 000	Seq Scan → Sort , 3,46 мс	Bitmap → Sort , 0,055 мс	Bitmap по (user_id) → Sort , 0,034 мс	—
1 000 000	Parallel Seq Scan → Sort , 19,8 мс	Bitmap → Sort , 0,057 мс	Bitmap по составному → Sort , 0,063 мс	—
5 000 000	Parallel Seq Scan → Sort , 90,3 мс	Bitmap → Sort , 0,134 мс	Bitmap по составному → Sort , 0,121 мс	—
10 000 000	Parallel Seq Scan → Sort , 165,9 мс	Bitmap → Sort , 0,166 мс	Bitmap по составному → Sort , 0,166 мс	Index Scan, без Sort , 0,14–0,32 мс

1) **Изменился ли план?** Индекс по user_id убрал полный проход, но оставил Sort. После создания составного индекса при 100 тыс.–10 млн строк планировщик выбрал **Bitmap Heap Scan + Sort**. У пользователя 123 всего ~100 строк при LIMIT 100: прочитать их вразнобой и отсортировать 100 значений в памяти он считает дешевле, чем идти по индексу в порядке ключа.

2) **Исчезла ли операция Sort?** Когда в таблице остался только составной индекс (10 млн строк, задание 11), да:

Листинг 6 – План выполнения (задание 7)

```
Limit (actual time=0.037..0.096 rows=100 loops=1)
-> Index Scan using idx_events_user_created on events (actual time=0.036..0.087 rows=100 loops=1)
    Index Cond: (user_id = 123)
Execution Time: 0.138 ms
```

Выигрыш здесь мал, потому что сортировать приходится всего ~100 строк. В ЛР №1 для пользователя с 50 000 заказов тот же приём дал 30 мс → 0,09 мс: Sort и чтение всех строк пользователя исчезают, и работа ограничивается LIMIT.

3) **Почему важен порядок колонок?** B-tree (user_id, created_at DESC) хранит записи сгруппированными по user_id, а внутри группы — уже по убыванию даты. Поэтому для WHERE user_id = 123 ORDER BY created_at DESC нужные записи лежат одним отрезком в нужном порядке. В индексе (created_at, user_id) записи пользователя разбросаны по всему индексу, и сортировка никуда не денется.

1.7 Задание 8. Агрегация

Листинг 7 – SQL-запрос (задание 8)

```
SELECT event_type, COUNT(*) FROM events
WHERE created_at >= NOW() - INTERVAL '30 days'
GROUP BY event_type;
```

Таблица 9 – Агрегация за 30 дней

Объём	Строк за 30 дней	Без индекса	С индексом created_at	Выигрыш
10 000	819	1,43 мс	0,32 мс	в 4,4 раза
100 000	8 400	14,5 мс	3,28 мс	в 4,4 раза
1 000 000	82 371	60,2 мс	35,3 мс	в 1,7 раза
5 000 000	410 391	284,1 мс	147,8 мс	в 1,9 раза
10 000 000	820 827	549,2 мс	482,0 мс	в 1,1 раза

1) **Какие операции в плане?** С 1 млн строк: Finalize GroupAggregate → Gather Merge → Sort → Partial HashAggregate → Parallel Seq Scan (без индекса) или → Bitmap Heap Scan → Bitmap Index Scan (с индексом). Каждый процесс считает частичные суммы, затем они сливаются.

2) **Какой объём нужно обработать?** Все строки за 30 дней — 8,2 % таблицы: 820 тыс. строк на 10 млн. Сократить это количество нельзя: каждую строку надо посчитать.

3) **Помогает ли индекс на created_at?** Частично. Он избавляет от чтения остальных 92 % строк, но выигрыш тает с ростом объёма: в 4 раза на 100 тыс., в 1,1 раза на 10 млн. 820 тыс. найденных строк разбросаны по 85 тыс. страниц, то есть почти по всей таблице.

4) **Может ли индекс полностью решить проблему?** Нет. Время растёт пропорционально числу строк в окне (0,5 с на 10 млн, ~5 с на 100 млн), и индекс лишь меняет способ их чтения. Проблему решает уменьшение объёма обрабатываемых данных: предагрегация, секционирование по времени (задание 11).

1.8 Задание 9. Стоимость индексов

Одинаковые данные (setseed) вставлялись в таблицу, где уже лежит 1 млн строк, — сначала только с РК, затем с тремя индексами из задания.

Таблица 10 – Операции записи без дополнительных индексов и с тремя индексами

Операция	Только РК	РК + 3 индекса	Во сколько раз
INSERT 1 000 000 строк	4,72 с	12,39 с	в 2,6 раза
UPDATE 100 000 строк (меняется <code>created_at</code>)	0,55 с	2,17 с	в 4,0 раза
DELETE 100 000 строк	0,19 с	0,05 с	не зависит от индексов
Размер индексов после операций (таблица 137 МБ)	45 МБ	206 МБ	в 4,6 раза

1) **Как изменилось время INSERT?** Выросло в 2,6 раза, UPDATE индексируемой колонки — в 4 раза.

2) **Почему индексы замедляют запись?** Каждая вставленная строка — это запись не в одну структуру, а в четыре: таблицу, РК и три индекса. Индексы по `user_id` и `created_at` принимают новые записи в случайные места дерева, что даёт промахи кэша, деление страниц и дополнительный WAL.

3) **Что происходит с индексами при добавлении строк?** Для каждого индекса вычисляется ключ, выполняется спуск по B-tree до нужного листа и вставляется пара «ключ → адрес строки». При переполнении лист делится. UPDATE в PostgreSQL создаёт новую версию строки, поэтому новые записи попадают во все индексы. DELETE только помечает строку удалённой, индексы чистит потом VACUUM, поэтому удаление от индексов почти не зависит.

Почему нельзя создать индекс на каждой колонке? Индексы замедляют каждую запись, занимают столько же места, сколько данные (задание 10), и помогают только тем запросам, условия которых совпадают с их ключом. Индекс на каждой колонке заставит платить при каждой вставке за индексы, которыми почти никто не пользуется.

1.9 Задание 10. Размер индексов

Таблица 11 – Размер индексов и время их построения

Объём	idx_events_user_id	idx_events_created_at	idx_events_user_created
10 000	240 кБ (8 мс)	240 кБ (13 мс)	328 кБ (8 мс)
100 000	1,9 МБ (41 мс)	2,2 МБ (44 мс)	3,0 МБ (51 мс)
1 000 000	8,9 МБ (0,47 с)	21,4 МБ (0,42 с)	30,1 МБ (0,50 с)
5 000 000	33,9 МБ (1,84 с)	107,1 МБ (2,20 с)	150,4 МБ (2,94 с)
10 000 000	67,7 МБ (3,47 с)	214,2 МБ (6,39 с)	300,8 МБ (5,14 с)

Как размер индекса зависит от числа записей? Линейно, но с разным коэффициентом:

- created_at — ~22,5 байта на строку. Значения почти уникальны: ключ 8 байт, адрес строки 6 байт, служебные данные и заполнение страниц;

- (user_id, created_at DESC) — ~31,5 байта на строку, ключ длиннее;

- user_id — всего ~7 байт на строку, в три раза меньше created_at при том же размере ключа. Это дедупликация B-tree (PostgreSQL 13+): у 100 тыс. различных user_id на 10 млн строк примерно по 100 повторов, и повторяющийся ключ хранится один раз со списком адресов строк. На 10–100 тыс. строк повторов почти нет, и индексы user_id и created_at одинаковы по размеру.

Время построения тоже растёт линейно (с учётом сортировки — чуть быстрее): ~0,4–0,6 с на 1 млн строк. На растущей таблице индекс надо строить CONCURRENTLY, иначе запись блокируется на секунды и минуты.

1.10 Задание 11. Когда индекс уже не спасает

Листинг 8 – SQL-запрос (задание 11)

```
SELECT DATE(created_at), COUNT(*) FROM events
WHERE created_at >= NOW() - INTERVAL '365 days'
GROUP BY DATE(created_at);
```

Таблица 12 – Агрегация за год на разных объёмах

Объём	Без индекса	С индексом created_at	План
10 000	3,4 мс	4,6 мс	Seq Scan → HashAggregate
100 000	31,0 мс	30,9 мс	Seq Scan → HashAggregate

Объём	Без индекса	С индексом <code>created_at</code>	План
1 000 000	291,9 мс	286,0 мс	Seq Scan → HashAggregate
5 000 000	1 436,0 мс	1 443,4 мс	Seq Scan → HashAggregate
10 000 000	2 829,9 мс	2 911,5 мс	Seq Scan → HashAggregate

Дополнительно на 10 млн строк:

Таблица 13 – Варианты выполнения агрегации за год на 10 млн строк

Вариант	План	Execution Time
Индекс <code>created_at</code> есть, VACUUM выполнен — выбор планировщика	Seq Scan → HashAggregate	2 967,9 мс
Только индекс (<code>enable_seqscan = off</code>)	Index Only Scan, Heap Fetches: 0 → HashAggregate	2 880,9 мс
Дневная сводка <code>events_daily</code> (1 464 строки, 192 кБ, строится 1,6 с)	Seq Scan по сводке → HashAggregate	0,58 мс

1) **Почему запрос дорогой при наличии индекса?** Условие «за 365 дней» выбирает всю таблицу (99,99 % строк). Индекс умеет быстро найти немногие строки, а здесь нужны все. Даже чтение одного индекса без таблицы (Index Only Scan) заняло те же 2,9 с: 10 млн записей индекса нужно перебрать и сгруппировать.

2) **Какой объём нужно обработать?** Все строки таблицы. На 10 млн это 650 МБ данных (или 214 МБ индекса), на 100 млн по измеренному линейному росту — около 6,5 ГБ и ~29 с на один запрос (2,9 с × 10).

3) **Почему индекс не гарантирует мгновенного выполнения?** Индекс ускоряет поиск, но не сокращает объём работы, которую требует сам запрос. Стоимость агрегации пропорциональна числу агрегируемых строк, и индекс его не меняет. Кроме того, на больших объёмах добавляются JIT-компиляция (в плане JIT: Total 66–174 ms) и работа с временными данными.

4) **Какие решения рассмотреть при дальнейшем росте:**

– **преагрегация** — дневная сводка или материализованное представление. Считать заранее, а при запросе читать 366 строк вместо 10 млн: 2 880 мс → 0,58 мс. Сводку дополняют по мере прихода событий или триггером;

- **секционирование по `created_at`** — запрос за период читает только нужные секции, старые секции можно быстро удалять или архивировать;

- **BRIN-индекс** на `created_at` для таблиц, куда строки пишутся по времени: он в сотни раз меньше B-tree и хорош для диапазонов;

- **ограничение хранения** — архивирование старых событий;

- **отдельное аналитическое хранилище** (колоночная СУБД, TimescaleDB с continuous aggregates) и кэширование готовых отчётов.

Проблема инфраструктуры на больших объёмах. Первый запуск задания 11 упал с ошибкой `could not resize shared memory segment ... No space left on device`. Параллельному `VACUUM` таблицы из 10 млн строк с индексами понадобилось 64 МБ разделяемой памяти, а Docker по умолчанию выделяет контейнеру `/dev/shm` ровно 64 МБ. В `docker-compose.yml` добавлен `shm_size: 512mb`.

2 Исследование собственного сервиса TaskManager

2.1 Задание 12. Выбор сущности

- **Сущность:** таблица `tasks` сервиса TaskManager (задачи пользователей). Она же основная сущность для масштабирования в ЛР №1.
- **Количество записей:** до начала работы — 1 000 000 сгенерированных задач. В ходе работы таблица выращена ступенями до 100 тыс., 1 млн и 5 млн, сейчас в ней **5 000 000** задач и ~4 млн связей задача–тег.
- **Почему выбрана:** все основные запросы API работают с задачами (`GET /api/tasks` с фильтрами, сортировкой и пагинацией, `GET /api/tasks/stats`), остальные таблицы — справочные.
- **Почему растёт быстрее остальных:** задачи создаются постоянно каждым пользователем и не удаляются после выполнения. Объём растёт как «пользователи × время» (~7,5 млн строк в год при 10 тыс. активных пользователей, оценка из ЛР №1), вместе с ним растёт `task_tags`.

2.2 Задание 13. Выбор запросов

Запросы сняты из лога PostgreSQL: это ровно тот SQL, который EF Core генерирует для API (`docs/lab-02/results/b0-captured-due-date-sql.txt`, `docs/lab-01/results/part2-01-captured-sql.txt`). Все они воспроизводятся скриптом `scripts/lab-02/b1-measure.sql`.

Листинг 9 – Запрос 1 — поиск по внешнему ключу (`GET /api/tasks` обычного пользователя, число задач для пагинации)

```
SELECT count(*)::int FROM tasks AS t WHERE t.user_id = $1
```

Листинг 10 – Запрос 2 — диапазон дат (`GET /api/tasks?dueDateFrom=...&dueDateTo=...`, администратор) — число задач и страница

```
SELECT count(*)::int FROM tasks AS t
WHERE t.due_date >= $1 AND t.due_date <= $2;

SELECT t0.*, p.*, u.*, t1.*           -- в оригинале перечислены все колонки
FROM (
```

```

SELECT t.* FROM tasks AS t
WHERE t.due_date >= $1 AND t.due_date <= $2
ORDER BY t.created_at DESC
LIMIT $3 OFFSET $4
) AS t0
LEFT JOIN projects AS p ON t0.project_id = p.id
INNER JOIN users AS u ON t0.user_id = u.id
LEFT JOIN (SELECT ... FROM task_tags AS t2 INNER JOIN tags AS t3 ON t2.tag_id = t3.id) AS t1
ON t0.id = t1.task_id
ORDER BY t0.created_at DESC, t0.id, p.id, u.id, t1.task_id, t1.tag_id;

```

Запрос 3 — фильтрация + сортировка + ограничение (GET /api/tasks, страница активного пользователя gen_user_1, у которого 1 % всех задач): тот же запрос с WHERE t.user_id = \$1 ORDER BY t.created_at DESC LIMIT 10.

Дополнительно измерялись:

– **запрос 4** — GET /api/tasks/stats администратора: SELECT t.status, count(*), count(*) FILTER (WHERE ...) FROM tasks AS t GROUP BY t.status;

– **запрос 5** — SELECT count(*) FROM tasks для пагинации администратора;

– **запрос 6** — страница всех задач без фильтров (контроль после изменения индексов).

Найденная ошибка. При подготовке запроса 2 выяснилось, что фильтр по сроку в API не работал: GET /api/tasks?dueDateFrom=2026-01-01 возвращал 500. Дата из строки запроса приходит без часового пояса (Kind=Unspecified), а Npgsql отказывается записывать такую дату в timestamp with time zone. Исправлено в TaskService (даты приводятся к UTC) и закрыто unit-тестом TaskServiceTests.

2.3 Задание 14. Baseline

Базовое измерение — на 1 млн задач со схемой после ЛР №1. Полные планы — docs/lab-02/results/bl-growth.txt.

Таблица 14 – Базовое измерение на 1 000 000 задач

Запрос	Основные операции плана	Индексы	Строк прочитано	Execution Time
1. count по user_id	Aggregate → Index Only Scan	idx_tasks_user_id_created_at	9 996	1,26 мс

Запрос	Основные операции плана	Индексы	Строк прочитано	Execution Time
2. count по сроку	Aggregate → Index Only Scan	idx_tasks_due_date	6 846	0,98 мс
2. страница по сроку	Limit → Index Scan + Filter (Rows Removed 238 759) → JOIN	idx_tasks_created_at	238 769	188,3 мс
3. страница пользователя	Limit → Index Scan → JOIN	idx_tasks_user_id_created_at	10	0,39 мс
4. статистика	Gather Merge → Partial HashAggregate → Parallel Seq Scan	—	999 999	86,0 мс
5. count всех задач	Gather → Parallel Index Only Scan	idx_tasks_priority	999 999	38,4 мс

2.4 Задание 15. Увеличение объёма данных

Генератор `scripts/generate-data.sql` доработан так, что повторный запуск дописывает задачи, не дублируя пользователей, проекты и теги. Скрипт `run-part-b.sh` очищает `tasks` и выращивает её ступенями.

Таблица 15 – Рост таблицы `tasks`

Ступень	Вставка задач	Вставка связей с тегами	Таблица	Индексы (7 шт.)	Всего
0 → 100 000	7,0 с	4,7 с	13,5 МБ	16,2 МБ	29,8 МБ
100 000 → 1 000 000	58,7 с	44,2 с	135,1 МБ	162,6 МБ	297,7 МБ
1 000 000 → 5 000 000	300,7 с	258,8 с	686,8 МБ	803,6 МБ	1,46 ГБ

Одна задача вставляется за ~70 мкс — в 28 раз дольше, чем строка `events` без индексов (2,5 мкс, задание 2). Помимо генерации UUID и соединений в генераторе, при каждой вставке обновляются семь индексов `tasks`, включая первичный ключ по случайному UUID. Индексы `tasks` уже весят больше самой таблицы.

2.5 Задание 16. Сравнение производительности

Таблица 16 – Время выполнения запросов при росте `tasks`

Запрос	Объём	Execution Time	Scan	Индекс	Строк прочитано
№1 count по <code>user_id</code>	100 000	0,24 мс	Index Only Scan	idx_tasks_user_id_created_at	978
№1	1 000 000	1,26 мс	Index Only Scan	idx_tasks_user_id_created_at	9 996
№1	5 000 000	6,30 мс	Index Only Scan	idx_tasks_user_id_created_at	49 874

Запрос	Объём	Execution Time	Scan	Индекс	Строк прочитано
№2 count по сроку	100 000	0,12 мс	Index Only Scan	idx_tasks_due_date	698
№2	1 000 000	0,98 мс	Index Only Scan	idx_tasks_due_date	6 846
№2	5 000 000	6,27 мс	Index Only Scan	idx_tasks_due_date	34 269
№2 страница по сроку	100 000	9,8 мс	Index Scan + Filter	idx_tasks_created_at	24 327
№2	1 000 000	188,3 мс	Index Scan + Filter	idx_tasks_created_at	238 769
№2	5 000 000	2 820,2 мс	Index Scan + Filter	idx_tasks_created_at	1 189 544
№3 страница пользователя	100 000	0,34 мс	Index Scan	idx_tasks_user_id_created_at	10
№3	1 000 000	0,39 мс	Index Scan	idx_tasks_user_id_created_at	10
№3	5 000 000	0,45 мс	Index Scan	idx_tasks_user_id_created_at	10
№4 статистика	100 000	17,7 мс	Seq Scan	—	100 000
№4	1 000 000	86,0 мс	Parallel Seq Scan	—	999 999
№4	5 000 000	403,3 мс	Parallel Seq Scan	—	5 000 001
№5 count всех	100 000	10,3 мс	Index Only Scan	idx_tasks_status	100 000
№5	1 000 000	38,4 мс	Parallel Index Only Scan	idx_tasks_priority	999 999
№5	5 000 000	178,6 мс	Parallel Index Only Scan	idx_tasks_status	5 000 001

При росте данных в 50 раз (100 тыс. → 5 млн) запросы ведут себя по-разному:

- **страница пользователя (№3)** почти не меняется (в 1,3 раза): составной индекс из ЛР №1 и LIMIT ограничивают работу десятью строками;
- **счётчики (№1, №2 count, №5) и статистика (№4)** дорожают в 17–51 раз, пропорционально числу строк, которые нужно посчитать;
- **страница с фильтром по сроку (№2)** замедлилась в **288 раз** — быстрее, чем растут данные.

2.6 Задание 17. Узкое место

Сильнее всего от роста данных зависит **запрос 2 — страница задач с фильтром по сроку**: 9,8 мс → 188 мс → 2,8 с.

Листинг 11 – План на 5 млн задач (фрагмент, полный — в b1-growth.txt)

```
Limit (cost=0.43..156.10 rows=10 width=128) (actual time=2814.562..2819.446 rows=10 loops=1)
  Buffers: shared hit=216898 read=977020
```

```
-> Index Scan using idx_tasks_created_at on tasks t (cost=0.43..522854.55 rows=33588
width=128)
                                (actual time=2814.561..2819.438 rows=10
loops=1)
    Filter: ((due_date >= '2026-03-14 00:00:00+00') AND (due_date <= '2026-03-21
00:00:00+00'))
    Rows Removed by Filter: 1189534
Execution Time: 2820.195 ms
```

Проблемное место: Index Scan using idx_tasks_created_at с Filter по сроку и Rows Removed by Filter: 1 189 534. Чтобы найти 10 задач, PostgreSQL прочитал 1,19 млн строк и 1,19 млн страниц, из них 977 тыс. — с диска.

Почему запрос становится дорогим. Индекса, который одновременно фильтрует по due_date и отдаёт строки в порядке created_at, нет. У планировщика два пути:

- 1) найти все ~34 тыс. задач окна по idx_tasks_due_date и отсортировать их;
- 2) идти по idx_tasks_created_at от новых задач к старым и проверять срок, пока не наберётся 10 строк.

Планировщик считает колонки независимыми. По его оценке, подходящие задачи (0,7 % таблицы) равномерно разбросаны по времени создания, и 10 штук встретятся примерно через 1 500 строк: стоимость 156 против 522 855 за весь индекс. Но срок жёстко связан с датой создания (due_date = created_at + 0...90 дней). Задачи со сроком полгода назад созданы 6–9 месяцев назад, поэтому сначала приходится пропустить все задачи, созданные позже, — четверть таблицы. Их число растёт вместе с таблицей. Когда рабочий набор перестаёт помещаться в память, добавляется чтение с диска, поэтому рост быстрее линейного.

2.7 Задание 18. Попытка улучшить запрос

На 5 млн задач проверены три способа (scripts/lab-02/b2-fixes.sql, docs/lab-02/results/b2-fixes.txt):

Таблица 17 – Варианты ускорения запроса 2 на 5 000 000 задач

Вариант	Что меняется	План по tasks	Buffers	Execution Time
Исходный	—	Index Scan idx_tasks_created_at + Filter (1,19 млн строк)	1 194 006	2 820,2 мс
А. Составной индекс (created_at DESC, due_date)	индекс	Index Scan, срок проверяется в Index Cond — таблица не читается	4 657	20,9 мс
В. Сначала фильтр, потом сортировка (OFFSET 0)	SQL	Bitmap Index Scan idx_tasks_due_date (34 269 строк) → top-N Sort	28 654	108,6 мс
С. Сортировка по сроку (?sort=due_date)	порядок выдачи	Index Scan idx_tasks_due_date → Incremental Sort, 11 строк	108	0,51 мс

Выбран вариант А. Он ускоряет запрос в 135 раз, не меняя ни SQL, ни поведения API. Индекс (created_at DESC, due_date) начинается с той же колонки, что и idx_tasks_created_at, поэтому полностью его заменяет: список задач без фильтра (запрос 6) продолжает работать через новый индекс, и число индексов на таблице не растёт.

Изменение внесено миграцией docker/liquibase/changesets/011-create-tasks-created-at-due-date-index.xml (приложение Б): CREATE INDEX CONCURRENTLY нового индекса и DROP INDEX CONCURRENTLY старого, с блоком rollback. Миграция применяется автоматически при docker compose up. Откат (liquibase rollback-count 1) и повторное применение проверены на 5 млн задач.

Листинг 12 – План после миграции

```

Limit (cost=0.43..72.98 rows=10 width=128) (actual time=17.972..18.019 rows=10 loops=1)
  Buffers: shared hit=4569
  -> Index Scan using idx_tasks_created_at_due_date on tasks t (cost=0.43..237703.56
rows=32767 width=128)
                                                                    (actual time=17.971..18.015
rows=10 loops=1)
    Index Cond: ((due_date >= '2026-03-14 00:00:00+00') AND (due_date <= '2026-03-21
00:00:00+00'))
Execution Time: 18.548 ms

```

Таблица 18 – Запросы сервиса на 5 000 000 задач до и после миграции 011

Запрос	До	После	Индекс после
№2 страница по сроку	2 820,2 мс, Rows Removed 1 189 534	18,6 мс, Rows Removed 0	idx_tasks_created_at_du e_date
№6 страница без фильтров	0,37 мс	0,38 мс	idx_tasks_created_at_du e_date
№1 count по user_id	6,30 мс	6,39 мс	без изменений
№2 count по сроку	6,27 мс	6,15 мс	без изменений

Запрос	До	После	Индекс после
№3 страница пользователя	0,45 мс	0,42 мс	без изменений
№4 статистика	403,3 мс	407,2 мс	без изменений
№5 count всех	178,6 мс	179,3 мс	без изменений

Время ответа API на 5 млн задач (scripts/lab-02/api-timing.sh: вызов после прогрева EF Core, кэш Redis очищен). Состояние «до» получено откатом миграции через Liquibase (docs/lab-02/results/b6-rollback.txt):

Таблица 19 – Время ответа API на 5 000 000 задач до и после миграции 011

Endpoint	До миграции 011	После миграции 011
GET /api/tasks?dueDateFrom=...&dueDateTo=... (Admin)	2,95 с	0,060 с
GET /api/tasks?dueDateFrom=...&dueDateTo=...&sort=due_date (Admin)	0,029 с	0,027 с
GET /api/tasks (Admin, без фильтров)	0,103 с	0,089 с
GET /api/tasks (gen_user_1)	0,017 с	0,015 с
GET /api/tasks/stats (Admin)	0,324 с	0,324 с

Что осталось.

– Вариант А ускорил запрос, но не сделал его независимым от объёма: индекс по-прежнему перебирает все задачи новее окна, просто без чтения таблицы (4 657 страниц индекса на 5 млн).

– Для экранов «задачи по срокам» правильный вариант С: он уже доступен через ?sort=due_date и занимает 0,5 мс на любом объёме.

– Запросы 4 и 5 (статистика и общий счётчик) индексом не решаются. Для статистики проверен другой способ получения данных — материализованное представление: чтение 4 строк занимает 0,02 мс против 403 мс, обновление (REFRESH) — 312 мс, его можно выполнять по расписанию.

2.8 Задание 19. Граница применимости индексов

Когда индексов становится недостаточно — по измерениям этой работы:

При объёме около 1 млн задач запрос `GET /api/tasks` с фильтром по сроку начинает перебирать сотни тысяч строк по индексу `created_at` с проверкой срока в таблице (Rows Removed by Filter 238 759) и занимает 188 мс. При 5 млн задач — 1,19 млн строк и 2,8 с. Составной индекс (`created_at DESC, due_date`) уменьшил время до 18,6 мс. Однако запрос по-прежнему просматривает все записи индекса, созданные после окна сроков, а их число растёт вместе с таблицей: при линейном росте это ~190 мс на 50 млн задач. Поэтому для этого сценария одного индекса недостаточно: нужно менять запрос (сортировка по сроку — 0,5 мс, не зависит от объёма) или объём просматриваемых данных (секционирование по `created_at`, архив завершённых задач).

Та же граница видна в остальных измерениях:

- **агрегаты по всей таблице** (статистика задач — 403 мс на 5 млн, агрегация событий за год — 2,9 с на 10 млн) не ускоряются никаким индексом, даже Index Only Scan: объём работы задаёт сам запрос. Решение — преагрегация (0,02–0,58 мс);

- **индексный поиск небольшой части таблицы** (поиск по `user_id`, страница пользователя) остаётся быстрым на всех объёмах: 0,16 мс на 10 млн событий, 0,45 мс на 5 млн задач.

Индекс спасает, пока запрос ограничивает объём работы размером результата. Как только работа зависит от размера таблицы — агрегат, большая доля строк, фильтр, не совпадающий с порядком индекса, — индексов становится недостаточно, и нужно менять запрос, модель данных или способ хранения.

3 Ответы на контрольные вопросы

1) **Почему Seq Scan не всегда плохой план?** Когда нужна большая часть таблицы, последовательное чтение всех страниц дешевле, чем тот же объём вразнобой через индекс. Выборку за полгода (49 % строк) планировщик на всех объёмах выполнял Seq Scan и с индексом, и без него, с одинаковым временем. На маленькой таблице (10 тыс. строк) Seq Scan занимает доли миллисекунды.

2) **Почему PostgreSQL может не использовать индекс?** Причины:

- условие выбирает большую долю строк;
- по оценке стоимости другой план дешевле: в задании 7 при наличии составного индекса выбран Bitmap + Sort по 100 строкам;
- выражение в условии не совпадает с ключом индекса;
- статистика устарела или ошибается (часть B, запрос 2).

3) **Что происходит с индексом при INSERT?** Для каждой новой строки вычисляется ключ, выполняется спуск по B-tree до нужного листа и туда добавляется пара «ключ → адрес строки». При переполнении страница делится, и всё это пишется в WAL.

4) **Почему много индексов замедляет запись?** Каждый индекс — отдельная структура, которую нужно обновить при каждой вставке и при изменении индексируемой колонки. Три индекса замедлили INSERT в 2,6 раза, UPDATE — в 4 раза (задание 9).

5) **Что такое селективность?** Доля строк, удовлетворяющих условию. `user_id = 123` на 10 млн строк — 103 строки (0,001 %, высокая селективность), события за сутки — 0,27 %, за полгода — 49 % (низкая).

6) **Почему индекс хорош для небольшой части таблицы?** Поиск по B-tree стоит логарифм от числа строк плюс чтение найденных строк. Для 103 строк из 10 млн это ~100 страниц вместо 83 тыс., поэтому 161 мс превращаются в 0,16 мс.

7) **Почему индекс менее полезен, если запрос возвращает большую часть таблицы?** Каждая найденная строка — случайное обращение к странице таблицы, а при большой доле строк это почти все страницы. К ним добавляется чтение самого индекса. Последовательное чтение таблицы оказывается дешевле.

8) **Зачем нужен EXPLAIN ANALYZE?** Он выполняет запрос и показывает реальный план с фактическим числом строк и временем каждого узла. Только так можно подтвердить, что изменение действительно ускорило запрос, а не просто создало индекс.

9) **Что означает Rows Removed by Filter?** Число строк, которые узел прочитал и отбросил, потому что они не прошли условие `Filter`. На 10 млн строк без индекса — 9 999 897 отброшенных ради 102 найденных: признак того, что условие не поддержано индексом.

10) **Что означает Index Cond?** Условие, которое проверяется при поиске в индексе: по нему определяется, какой отрезок индекса читать. В отличие от `Filter`, строки, не прошедшие `Index Cond`, вообще не читаются.

11) **Чем Index Scan отличается от Bitmap Index Scan?** `Index Scan` идёт по индексу и сразу читает строку таблицы для каждой записи, в порядке индекса, прыжками по файлу. `Bitmap Index Scan` сначала собирает битовую карту адресов всех подходящих строк, а `Bitmap Heap Scan` читает страницы таблицы в физическом порядке, каждую по одному разу. `Bitmap` выгоднее для «средних» выборок, но теряет порядок индекса: для `ORDER BY` нужен `Sort`.

12) **Почему порядок колонок в составном индексе важен?** `B-tree` упорядочен по первой колонке, вторая упорядочена только внутри одинаковых значений первой. Поэтому колонки с условием равенства ставятся первыми, а колонка диапазона или сортировки — последней (задание 7).

13) **Почему GROUP BY может оставаться дорогим при наличии индекса?** Агрегация должна обработать каждую строку группы. Агрегация

за год на 10 млн строк заняла 2,9 с без индекса, с индексом и при чтении одного индекса (задание 11). Индекс сокращает поиск, но не объём агрегации.

14) Как статистика влияет на выбор плана? По статистике `ANALYZE` (частые значения, гистограммы, корреляция) планировщик оценивает, сколько строк вернёт каждое условие, и сравнивает стоимость планов. После каждой ступени роста выполнялся `VACUUM ANALYZE`. Если оценка неверна — например, планировщик считает колонки независимыми, хотя они связаны (часть В, запрос 2), — выбирается план, который на порядки медленнее ожидаемого.

15) Что происходит с производительностью при росте таблицы? Размер таблицы и индексов растёт линейно. Запросы без подходящего индекса дорожают линейно (0,3 мс → 161 мс на 10 тыс. → 10 млн строк). Запросы с индексом дорожают только вместе с размером результата. Агрегации и выборки большой доли таблицы дорожают линейно при любых индексах.

16) Почему нельзя создавать индексы на все колонки? Индексы замедляют каждую запись, занимают столько же места, сколько таблица (646 МБ индексов при 651 МБ данных), и используются только запросами с подходящими условиями.

17) Как определить размер таблицы? `pg_relation_size('events')` — только данные, `pg_table_size` — с `TOAST`, `pg_total_relation_size('events')` — вместе со всеми индексами; в `psql` — `\dt+`.

18) Как определить размер индекса?
`pg_relation_size('idx_events_user_id')` или
`pg_relation_size(indexrelid)` из `pg_stat_user_indexes`; суммарно — `pg_indexes_size('events')`; в `psql` — `\di+`.

ЗАКЛЮЧЕНИЕ

В работе измерено, как рост объёма данных влияет на работу PostgreSQL: на экспериментальной таблице `events` от 10 тысяч до 10 миллионов строк и на таблице `tasks` сервиса TaskManager от 100 тысяч до 5 миллионов задач.

Основные результаты части А:

- размер таблицы и индексов растёт линейно: 651 МБ данных и 646 МБ трёх индексов на 10 млн строк. Индекс по колонке с повторами (`user_id`) благодаря дедупликации B-tree в три раза меньше индекса по уникальным значениям;

- поиск без индекса дорожает пропорционально таблице (0,3 мс → 161 мс), с индексом — только вместе с результатом (0,16 мс на 10 млн, в 1 016 раз быстрее);

- выборка большой доли таблицы (за полгода) и агрегаты по всей таблице (за год: 2,9 с на 10 млн) индексом не ускоряются. Агрегацию ускорила только предагрегированная сводка — до 0,58 мс;

- три индекса замедляют вставку в 2,6 раза и обновление индексируемой колонки в 4 раза.

В части В на реальных запросах сервиса найдено узкое место: страница задач с фильтром по сроку замедлилась в 288 раз при росте данных в 50 раз и заняла 2,8 с на 5 млн задач. Планировщик считал срок и дату создания независимыми и перебирал четверть таблицы. Миграция 011 заменила индекс (`created_at DESC`) на (`created_at DESC, due_date`) и уменьшила время до 18,6 мс без роста числа индексов. Попутно исправлена ошибка, из-за которой фильтр по сроку в API возвращал 500, а в `docker-compose.yml` увеличена разделяемая память Postgres, которой не хватило параллельному `VACUUM` на 10 млн строк.

Главный вывод: индекс эффективен, пока объём работы запроса определяется размером результата. Когда запрос вынужден обработать долю

таблицы, растущую вместе с данными, индекс лишь откладывает проблему. Дальше нужны изменение запроса, агрегация или секционирование — это тема следующих работ модуля.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 PostgreSQL 16 Documentation. Chapter 11. Indexes [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/indexes.html> (дата обращения: 10.09.2026).

2 PostgreSQL 16 Documentation. 14.1. Using EXPLAIN [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/using-explain.html> (дата обращения: 10.09.2026).

3 PostgreSQL 16 Documentation. 67.4. B-Tree Implementation (Deduplication) [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/btree-implementation.html> (дата обращения: 10.09.2026).

4 PostgreSQL 16 Documentation. 5.11. Table Partitioning [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/ddl-partitioning.html> (дата обращения: 10.09.2026).

5 PostgreSQL 16 Documentation. 41.3. Materialized Views [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/rules-materializedviews.html> (дата обращения: 10.09.2026).

6 Лабораторная работа № 2. Когда индексов недостаточно: производительность PostgreSQL при росте данных : методические указания. – Сириус : Колледж Университета «Сириус», 2026. – 13 с.

7 VHS — инструмент записи терминала [Электронный ресурс]. – URL: <https://github.com/charmbracelet/vhs> (дата обращения: 10.09.2026).

ПРИЛОЖЕНИЕ А

Скрипт измерений одного объёма данных

Скрипт `scripts/lab-02/a1-step.sql` выполняется для каждого контрольного объёма таблицы `events`. Функция `lab02.measure` описана в `scripts/lab-02/a0-setup.sql`.

Листинг А.1 – `a1-step.sql`

```
-- JP №2, часть А: все измерения для одного объёма данных (psql-переменная :target).
-- Вызывается из a1-growth.sql. В конце шага дополнительные индексы удаляются, чтобы
-- следующий объём снова измерялся «без индекса» и вставка шла без их обслуживания.

\echo
\echo '===== Объём: ' :target 'строк'
SELECT lab02.grow(:target);
VACUUM (ANALYZE) lab02.events;
SELECT lab02.record_sizes(:target, 'только PK');

\echo '--- Без дополнительных индексов (задания 4, 6, 7, 8, 11)'
SELECT * FROM lab02.measure('A', :target, 'user_eq', 'без индекса',
    $$SELECT * FROM lab02.events WHERE user_id = 123$$);
SELECT * FROM lab02.measure('A', :target, 'day', 'без индекса',
    $$SELECT * FROM lab02.events WHERE created_at >= NOW() - INTERVAL '1 day'$$);
SELECT * FROM lab02.measure('A', :target, 'half_year', 'без индекса',
    $$SELECT * FROM lab02.events WHERE created_at >= NOW() - INTERVAL '180 days'$$);
SELECT * FROM lab02.measure('A', :target, 'order', 'без индекса',
    $$SELECT * FROM lab02.events WHERE user_id = 123 ORDER BY created_at DESC LIMIT 100$$);
SELECT * FROM lab02.measure('A', :target, 'agg30', 'без индекса',
    $$SELECT event_type, COUNT(*) FROM lab02.events WHERE created_at >= NOW() - INTERVAL '30
days' GROUP BY event_type$$);
SELECT * FROM lab02.measure('A', :target, 'agg365', 'без индекса',
    $$SELECT DATE(created_at), COUNT(*) FROM lab02.events WHERE created_at >= NOW() - INTERVAL
'365 days' GROUP BY DATE(created_at)$$);

\echo '--- Индексы (user_id) и (created_at) (задания 5, 6, 8)'
SELECT lab02.create_index(:target, 'idx_events_user_id', 'CREATE INDEX idx_events_user_id ON
lab02.events (user_id)');
SELECT lab02.create_index(:target, 'idx_events_created_at', 'CREATE INDEX idx_events_created_at
ON lab02.events (created_at)');
ANALYZE lab02.events;
SELECT * FROM lab02.measure('A', :target, 'user_eq', 'индексы',
    $$SELECT * FROM lab02.events WHERE user_id = 123$$);
SELECT * FROM lab02.measure('A', :target, 'day', 'индексы',
    $$SELECT * FROM lab02.events WHERE created_at >= NOW() - INTERVAL '1 day'$$);
SELECT * FROM lab02.measure('A', :target, 'half_year', 'индексы',
    $$SELECT * FROM lab02.events WHERE created_at >= NOW() - INTERVAL '180 days'$$);
SELECT * FROM lab02.measure('A', :target, 'order', 'индексы',
    $$SELECT * FROM lab02.events WHERE user_id = 123 ORDER BY created_at DESC LIMIT 100$$);
SELECT * FROM lab02.measure('A', :target, 'agg30', 'индексы',
    $$SELECT event_type, COUNT(*) FROM lab02.events WHERE created_at >= NOW() - INTERVAL '30
days' GROUP BY event_type$$);
SELECT * FROM lab02.measure('A', :target, 'agg365', 'индексы',
    $$SELECT DATE(created_at), COUNT(*) FROM lab02.events WHERE created_at >= NOW() - INTERVAL
'365 days' GROUP BY DATE(created_at)$$);

\echo '--- Составной индекс (user_id, created_at DESC) (задание 7)'
SELECT lab02.create_index(:target, 'idx_events_user_created',
    'CREATE INDEX idx_events_user_created ON lab02.events (user_id, created_at DESC)');
ANALYZE lab02.events;
SELECT * FROM lab02.measure('A', :target, 'order', 'составной индекс',
    $$SELECT * FROM lab02.events WHERE user_id = 123 ORDER BY created_at DESC LIMIT 100$$);

SELECT lab02.record_sizes(:target, 'PK + 3 индекса');
DROP INDEX lab02.idx_events_user_id, lab02.idx_events_created_at,
lab02.idx_events_user_created;
```


ПРИЛОЖЕНИЕ Б

Миграция с составным индексом для фильтра по сроку

Миграция Liquibase, добавленная в проект по результатам заданий 17–18. Применяется автоматически при запуске `docker compose up`, откат проверен командой `liquibase rollback-count`.

Листинг Б.1 – 011-create-tasks-created-at-due-date-index.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
  xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
    http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.25.xsd">

  <!--
    ЛП №2 (docs/lab-02-growth.md, задания 17-18).
    GET /api/tasks?dueDateFrom=...&dueDateTo=... выполняет
      WHERE due_date BETWEEN ... ORDER BY created_at DESC LIMIT 10.
    Планировщик идёт по индексу created_at от новых задач к старым и проверяет срок
    в таблице для каждой строки: на 5 млн задач – 1.19 млн строк и 2.8 с на страницу.
    Составной индекс (created_at DESC, due_date) позволяет проверять срок прямо в индексе,
    не читая таблицу: 21 мс. Он же обслуживает список без фильтра (ORDER BY created_at
DESC),
    поэтому заменяет idx_tasks_created_at и число индексов не растёт.
  -->
  <changeSet id="011-create-tasks-created-at-due-date-index" author="taskmanager"
runInTransaction="false">
    <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tasks_created_at_due_date ON tasks
(created_at DESC, due_date)</sql>
    <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_tasks_created_at</sql>

    <rollback>
      <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tasks_created_at ON tasks
(created_at DESC)</sql>
      <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_tasks_created_at_due_date</sql>
    </rollback>
  </changeSet>
</databaseChangeLog>
```